

SmartFuzz

Security Assessment Report

Target: http://testphp.vulnweb.com

Scan Date: 2026-05-18 10:02:34 UTC

Scan ID: a1b2c3d4-e5f6-7890-abcd-ef1234567890

1. Executive Summary

This report presents the findings of an automated security assessment conducted against **http://testphp.vulnweb.com** on 2026-05-18 10:02:34 UTC. SmartFuzz crawled the target, generated AI-assisted payloads, and injected them into 47 test cases across 2 form(s) and 4 GET parameter set(s). A total of **3 vulnerabilities** were identified.

3	1	2	0	0	2	47
Total Findings	Critical	High	Medium	Low	Forms Crawled	Payloads Fired

Overall Risk Rating: **Critical**

2. Findings

2.1 Summary Table

#	Vulnerability	Parameter	Severity	Evidence / Signature
1	SQLi	cat	Critical	MySQL syntax error
2	XSS	search	High	Script tag reflected
3	SSRF	url	High	AWS metadata IP in response

2.2 Detailed Findings

#1 — SQLi in cat [Critical]

URL	http://testphp.vulnweb.com/listproducts.php?cat=1+OR+1%3D1
Method	GET
Status	500
Resp Time	0.342s
Timestamp	2026-05-18 10:01:05

Payload Used:

```
1 OR 1=1
```

Response Snippet:

```
You have an error in your SQL syntax near '1 OR 1=1'
```

Remediation:

Use parameterized queries. Never concatenate user input into SQL strings.

#2 — XSS in search [High]

URL	http://testphp.vulnweb.com/search.php?search=alert(1)
Method	GET
Status	200
Resp Time	0.218s
Timestamp	2026-05-18 10:01:22

Payload Used:

```
alert(1)
```

Response Snippet:

```
<div>Results for: <script>alert(1)</script></div>
```

Remediation:

Encode all user output. Use Content-Security-Policy headers.

#3 — SSRF in url [High]

URL	http://testphp.vulnweb.com/fetch?url=http://169.254.169.254/
Method	POST
Status	200
Resp Time	1.05s
Timestamp	2026-05-18 10:01:48

Payload Used:

```
http://169.254.169.254/latest/meta-data/
```

Response Snippet:

```
ami-id instance-id hostname
```

Remediation:

Whitelist allowed domains. Block requests to internal IP ranges.

3. Remediation Reference

SQL Injection (SQLi)

Risk: Allows attackers to read, modify, or delete database contents. Can lead to full system compromise.

1. Use parameterized queries / prepared statements exclusively.
2. Apply an ORM (SQLAlchemy, Django ORM) that handles escaping automatically.
3. Enforce least-privilege database accounts — the app user should not have DROP/ALTER rights.
4. Enable a Web Application Firewall (WAF) as a secondary layer.
5. Log and alert on SQL errors in production.

Reference: OWASP Top 10 A03:2021 — Injection

Cross-Site Scripting (XSS)

Risk: Enables attackers to execute scripts in victims' browsers, steal cookies/sessions, or redirect users.

1. HTML-encode all user-controlled output before rendering.
2. Implement a strict Content-Security-Policy (CSP) header.
3. Use modern frameworks (React, Vue, Angular) that auto-escape by default.
4. Set HttpOnly and Secure flags on session cookies.
5. Validate and sanitize input on the server side.

Reference: OWASP Top 10 A03:2021 — Injection

Server-Side Request Forgery (SSRF)

Risk: Lets attackers reach internal services, cloud metadata APIs, and private networks through the server.

1. Whitelist allowed destination domains/IPs; reject everything else.
2. Block requests to 169.254.0.0/16 (AWS metadata), 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16.
3. Disable HTTP redirects, or re-validate the final destination URL.
4. Use a dedicated egress proxy that enforces the allowlist.
5. Avoid user-supplied URLs entirely where possible.

Reference: OWASP Top 10 A10:2021 — SSRF

4. Disclaimer

This report was generated automatically by SmartFuzz. Results should be reviewed and validated by a qualified security professional before acting upon them. False positives may be present. This tool is intended for use against systems you own or have explicit written authorisation to test. Unauthorised testing is illegal and unethical.

Generated by SmartFuzz on 2026-05-18 10:02:34 UTC